Labo 5: Dynamic parallelism and asynchronous operation
Geavanceerde computerarchitecturen

Jan Martens & Leon Cherchye

November 30, 2024



Session Date: 20 november, 2024

Class: Ma ELICT optie software-engineering

1 Introduction

In dit labo hebben we onderzocht hoe we een kernel efficiënt kunnen oproepen door het oproepende proces op de CPU niet te laten wachten op de resultaten van de GPU. Hierbij werd gebruik gemaakt van zowel synchrone als asynchrone kernel-uitvoering om de prestaties te vergelijken.

2 Implementatie van de kernels

De implementatie maakt gebruik van CUDA om verschillende reduceerbewerkingen zoals som, product, minimum en maximum te berekenen op een gegeven array.
De `reductionKernel` bevat shared memory voor efficiënte datatoegang en implementeert een parallel reduceerproces. Afhankelijk van de gewenste operatie wordt een specifieke logica geselecteerd in de kernel.
Een kernpunt van de implementatie is het gebruik van `atomicAdd`, `atomicMin` en `atomicMax` om thread-safe resultaten op te slaan in globale geheugenlocaties. Bovendien maakt het programma gebruik van CUDA-streams voor asynchrone uitvoering van meerdere kernels. Voor de synchrone uitvoering is een andere kernel gemaakt.

3 Synchronous reduction: verschil in performance synchrone en asynchrone uitvoering

Bij synchrone uitvoering wacht de CPU op de voltooiing van elke kernel voordat deze verdergaat. Asynchrone uitvoering daarentegen maakt gebruik van CUDA-streams, waarmee meerdere kernels tegelijk kunnen worden uitgevoerd zonder dat de CPU telkens moet wachten.

3.1 Invloed van de input-grootte

De invloed van de input-grootte op de prestaties werd geëvalueerd door arrays van exponentieel toenemende groottes te gebruiken (tot 2^{24} elementen). Bij synchrone uitvoering neemt de rekentijd grofweg lineair toe met de input-grootte, zoals te zien in Figure 2 & Figure 3). In het geval van asynchrone uitvoering zien we een significante verbetering, vooral bij grotere arrays, doordat verschillende CUDA-streams de berekeningen kunnen overlappen. De uitvoeringstijd stijgt nog wel (zoals te zien in Figure 3), maar is veel kleiner waardoor het bij kleine array-groottes bijna niet waarneembaar is (Figure 1 & Figure 2).

3.2 Invloed van het aantal opgeroepen kernels

Het uitvoeren van meerdere kernels (voor som, product, minimum en maximum) werd getest. Bij synchrone uitvoering leidt dit tot wachttijden, aangezien kernels opeenvolgend worden uitgevoerd. Asynchrone uitvoering benut parallelisme door deze kernels simultaan te lanceren, wat resulteert in een aanzienlijke verkorting van de totale rekentijd. Hoe meer kernels tegelijk worden opgeroepen, hoe groter het verschil wordt tussen synchrone en assynchrone uitvoering. Dit is te zien in het verschil in uitvoeringstijd tussen de twee manieren in Figure 1 VS Figure 2.

Een afwijkend resultaat werd bekomen in een experiment met zeer grote array-sizes en 10 uitvoeringen van elke kernel (Figure 4). Hierbij zouden we verwachten dat de assynchrone uitvoering consistent sneller blijft dan de synchrone uitvoering, en zelfs verder zou uitlopen. Dit is echter

niet het geval: voor de grootste inputsize, is de synchrone kernel plots sneller dan de assynchrone. De verklaring hiervoor is moeilijk eenduidig vast te leggen, aangezien veel factoren simultaat invloed hebben op de uitvoeringstijd.

4 Conclusie

Dit experiment toont aan dat asynchrone kernel-uitvoering met CUDA-streams een duidelijke prestatiewinst biedt, vooral bij grotere input-groottes en meerdere kernels. Het efficiënt gebruik van GPU-middelen en het minimaliseren van CPU-wachttijden zijn hierbij cruciaal.

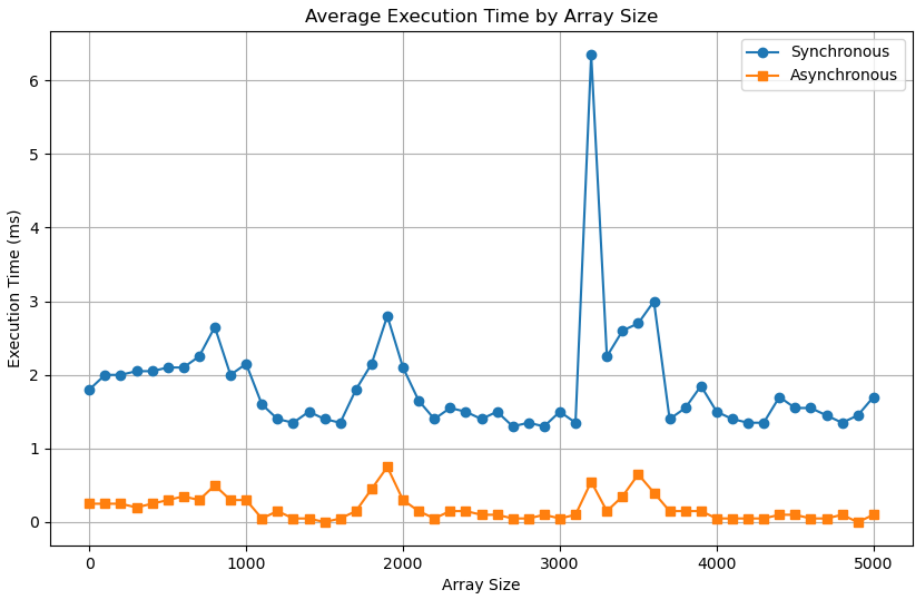


Figure 1: Kernel execution time vs input size (1 uitvoering van elke reductie)

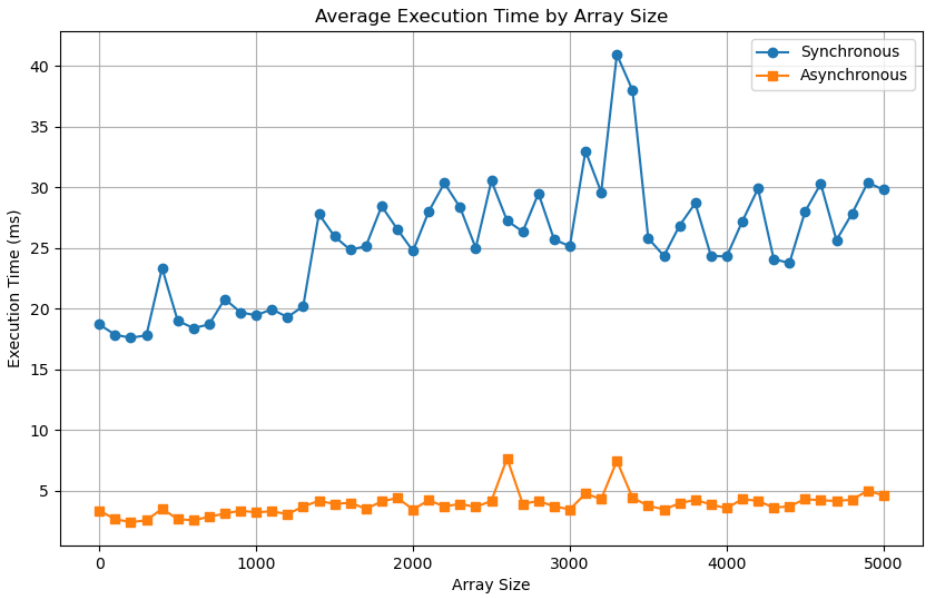


Figure 2: Kernel execution time vs input size (10 uitvoeringen van elke reductie)

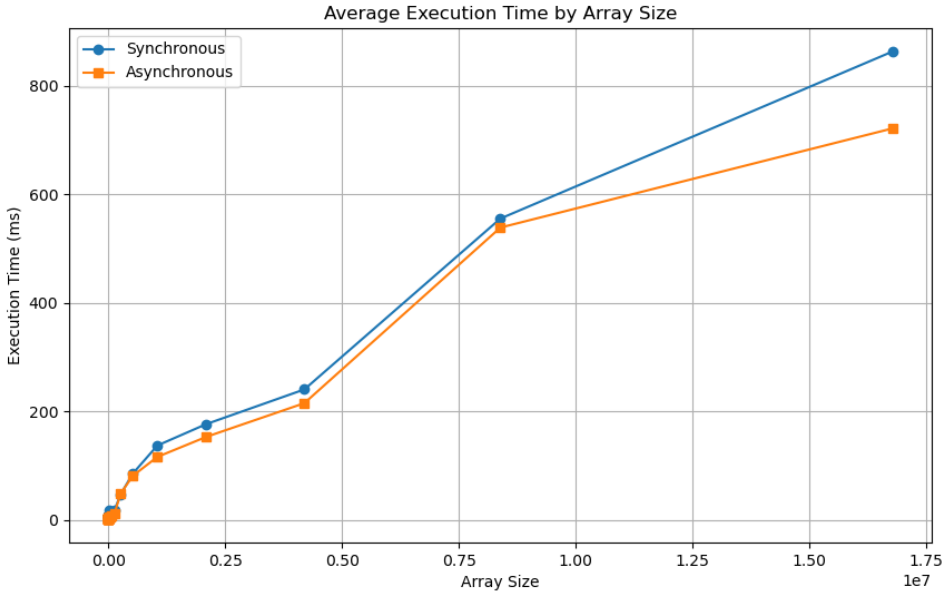


Figure 3: Kernel execution time vs input size (1 uitvoering van elke reductie) met grotere inputs

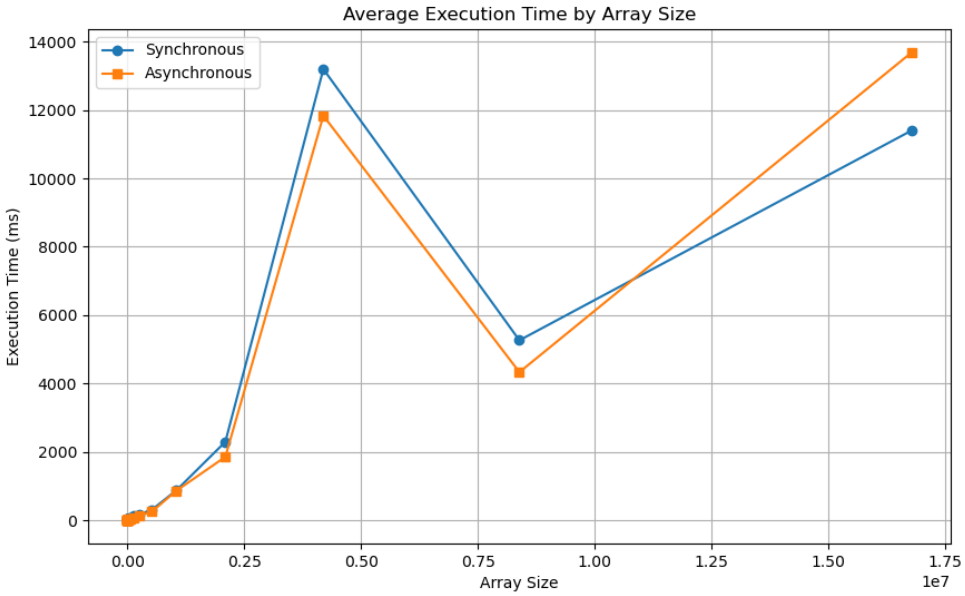


Figure 4: Kernel execution time vs input size (10 uitvoeringen van elke reductie) met grotere inputs